Algoritmo Dígito por Dígito para Cálculo de Raiz Quadrada em Ponto Fixo no ATmega328

Gabriel Luduvice Nunes Wolfovitch, João Pedro Nascimento Monteiro,
João Vítor de Araújo Costa de Oliveira e Lucas Miranda Campos
Departamento de Engenharia Elétrica e de Computação (DEEC)
Universidade Federal da Bahia (UFBA)
Salvador, Brasil

***Abstract*—Sistemas embarcados de instrumentação baseados em microcontroladores de 8 bits frequentemente necessitam linearizar grandezas físicas proporcionais ao quadrado do sinal medido, exigindo o cálculo eficiente da função raiz quadrada ($\sqrt{x}$). Restrições severas de memória e ausência de unidade de ponto flutuante (FPU) ou divisor em hardware no ATmega328 inviabilizam o uso de bibliotecas matemáticas convencionais. Este artigo apresenta a implementação e a caracterização técnica de um algoritmo iterativo de extração de raiz quadrada binária dígito a dígito (*shift-and-subtract* restaurador) desenvolvido em linguagem Assembly AVR. O sistema recebe radicandos inteiros sem sinal de 16 bits no formato Q16.0 e produz resultados em ponto fixo no formato Q8.6 após 14 iterações determinísticas. A análise numérica e arquitetural demonstra que a solução requer apenas 13 registradores da CPU, executa em aproximadamente 732 ciclos de clock ($45{,}75$ μs a 16 MHz) e garante erro absoluto estritamente limitado por um bit menos significativo (Erro $< 2^{-6} = 0{,}015625$).**

***Index Terms*—Raiz Quadrada, Ponto Fixo, Assembly AVR, ATmega328, Sistemas Embarcados, Análise de Erro Numérico.**

I. Introdução

Em sistemas embarcados de instrumentação — como medidores de vazão por pressão diferencial ou sensores de valor eficaz (RMS) —, a grandeza digitalizada pelo conversor analógico-digital é frequentemente proporcional ao quadrado da variável física de interesse ($x \propto y^2$). Para recuperar a grandeza original $y = \sqrt{x}$ em um microcontrolador de 8 bits como o ATmega328, o projeto enfrenta restrições arquiteturais importantes: a Unidade Lógica e Aritmética (ALU) opera nativamente sobre palavras de 8 bits, não dispõe de instruções de divisão em hardware e não possui unidade de ponto flutuante (FPU) dedicada [1].

A utilização de emulação de ponto flutuante em software (padrão IEEE 754) ou de métodos iterativos baseados em divisões sucessivas, como o método de Newton-Raphson, eleva substancialmente o consumo de memória de programa (Flash), o tempo de execução e a incerteza temporal (*jitter*). Para superar essas limitações, este trabalho documenta e analisa a implementação em Assembly AVR de um algoritmo de extração de raiz quadrada dígito a dígito em aritmética de ponto fixo, convertendo uma entrada inteira no formato **Q16.0** para uma saída fracionária no formato **Q8.6**.

II. A Técnica Utilizada e Seu Funcionamento

O algoritmo adotado baseia-se no método clássico de extração de raiz quadrada dígito a dígito na base 2 (radix-2), classificado como um algoritmo de deslocamento e subtração (*shift-and-subtract*) do tipo restaurador [2].

A. Fundamentação Matemática

Seja X o radicando inteiro de entrada e Q_k a aproximação parcial da raiz quadrada construída até a k-ésima iteração. A cada nova iteração, o algoritmo desloca a raiz parcial uma posição binária à esquerda e testa se o novo bit menos significativo $b_k \in \{0, 1\}$ deve ser igual a 1. Pela expansão binomial do quadrado de $(2Q_{k-1} + b_k)$, tem-se:

$$(2Q_{k-1} + b_k)^2 = 4Q_{k-1}^2 + (4Q_{k-1} + b_k)b_k \tag{1}$$

Se escolhermos $b_k = 1$, o incremento quadrático em relação ao quadrado parcial anterior ($4Q_{k-1}^2$) é dado pelo termo de teste T_k:

$$T_k = (2Q_{k-1} + 1)^2 - 4Q_{k-1}^2 = 4Q_{k-1} + 1 \tag{2}$$

Em representação binária, multiplicar Q_{k-1} por 4 e somar 1 equivale a deslocar Q_{k-1} dois bits para a esquerda e concatenar o par de bits $\texttt{01}$ nas duas posições menos significativas:

$$T_k = (Q_{k-1} \ll 2) \mid 01_2 \tag{3}$$

Paralelamente, o radicando X é consumido da esquerda para a direita (do bit mais significativo para o menos significativo) em grupos de 2 bits por iteração, pois $(2^1)^2 = 2^2 = 4$. O resto parcial A_k é atualizado deslocando-se o resto anterior dois bits à esquerda e inserindo os 2 bits mais significativos atuais de X:

$$A_k = (A_{k-1} \ll 2) \mid X_{\text{MSB}[15:14]} \tag{4}$$

Em seguida, calcula-se a diferença de teste $S_k = A_k - T_k$:

- **Caso $S_k \geq 0$:** O resíduo A_k comporta a subtração. O novo bit da raiz é $b_k = 1$, o resto parcial é atualizado ($A_k \leftarrow S_k$) e a raiz parcial passa a ser $Q_k = (Q_{k-1} \ll 1) \mid 1$.
- **Caso $S_k < 0$:** O termo de teste excede o resíduo disponível. O novo bit da raiz é $b_k = 0$, o resto parcial anterior A_k é preservado (caracterizando o método como *restaurador*) e a raiz parcial recebe apenas o deslocamento $Q_k = (Q_{k-1} \ll 1)$.

B. Extensão para Ponto Fixo (Q16.0 para Q8.6)

Para uma entrada X de 16 bits no formato inteiro Q16.0, as primeiras 8 iterações consomem integralmente os 8 pares de bits do radicando ($8 \times 2 = 16$ bits), produzindo os **8 bits da parte inteira** da raiz quadrada ($\lfloor \sqrt{X} \rfloor$).

Ao término da 8ª iteração, o registrador de entrada encontra-se completamente zerado. Contudo, caso ainda exista resto não nulo em A_8, a execução de iterações adicionais continua deslocando pares de bits `00` para dentro de A_k. Matematicamente, realizar F iterações extras após esgotar os bits de X equivale a multiplicar o radicando original por $4^F = 2^{2F}$ e extrair a raiz inteira:

$$Q_{8+F} = \left\lfloor \sqrt{X \cdot 2^{2F}} \right\rfloor = \left\lfloor 2^F \sqrt{X} \right\rfloor \tag{5}$$

Como a implementação executa um total de $N = 14$ iterações (8 inteiras $+6$ fracionárias, com $F = 6$), o valor binário final armazenado no registrador de saída está escalonado por $2^6 = 64$, correspondendo diretamente à representação em ponto fixo **Q8.6**.

III. Implementação em Assembly no ATmega328

Como a arquitetura AVR possui registradores de uso geral de 8 bits (`R0` a `R31`), todas as variáveis de 16 bits do caminho de dados foram implementadas agrupando pares de registradores adjacentes (*High:Low*), aproveitando instruções nativas de transferência de palavras de 16 bits como `MOVW` e operações aritméticas em cascata via *Carry flag*.

A. Alocação de Registradores

A Tabela I detalha o mapeamento dos registradores utilizado no programa `main.asm`. Ao todo, são empregados 13 registradores da CPU.

Table I
Mapeamento de Registradores na Implementação Assembly

Símbolo	Reg.	Função no Algoritmo
`INPUT`	`R19:R18`	Radicando de entrada X em Q16.0 (deslocado 2 bits/iteração)
`RESP`	`R21:R20`	Acumulador da raiz parcial Q_k e saída final em Q8.6
`A`	`R23:R22`	Resto parcial de trabalho A_k
`SINAL`	`R25:R24`	Diferença temporária $S_k = A_k - T_k$
`RESP_TEMP`	`R27:R26`	Termo de teste $T_k = (Q_{k-1} \ll 2) \mid 1$
`R28, R29`	`R28, R29`	Contadores dos sub-laços de deslocamento duplo ($2\times$)
`R30`	`R30`	Contador principal de iterações ($N =$ 0x0E $=$ 14)
`R16`	`R16`	Inicialização do ponteiro de pilha (`SPH:SPL`)

B. Estrutura das Sub-rotinas e Caminho de Dados

A Listagem 1 apresenta o núcleo funcional do código Assembly implementado para o ATmega328.

Listing 1. Nucleo do algoritmo em Assembly AVR (main.asm).

```
; Sub-rotina de teste e atualizacao da raiz
CALCULA_SINAL:
  MOVW RESP_TEMP_HIGH:RESP_TEMP_LOW, RESP_HIGH:RESP_LOW
  LDI  R28, 0x02      ; Contador p/ {RESP, 01}
SHIFT_SINAL:
  LSL  RESP_TEMP_LOW
  ROL  RESP_TEMP_HIGH
  DEC  R28
  BRNE SHIFT_SINAL

  SBR  RESP_TEMP_LOW, 0x01
  MOVW SINAL_HIGH:SINAL_LOW, A_HIGH:A_LOW
  SUB  SINAL_LOW, RESP_TEMP_LOW
  SBC  SINAL_HIGH, RESP_TEMP_HIGH

  BRMI NEGATIVO       ; Desvia se S_k < 0
  LSL  RESP_LOW
  ROL  RESP_HIGH
  MOVW A_HIGH:A_LOW, SINAL_HIGH:SINAL_LOW
  SBR  RESP_LOW, 0x01
  RET
NEGATIVO:
  LSL  RESP_LOW
  ROL  RESP_HIGH
  RET

; Sub-rotina de deslocamento duplo (INPUT -> A)
SHIFT_DOIS:
  LDI  R29, 0x02      ; Contador p/ 2 shifts
SHIFT:
  LSL  INPUT_LOW
  ROL  INPUT_HIGH
  ROL  A_LOW
  ROL  A_HIGH
  DEC  R29
  BRNE SHIFT
  RET
```

O funcionamento coordenado divide-se em três blocos principais:

1) **Inicialização e Controle (`MAIN` e `LOOP`):** O ponteiro de pilha (`SPH:SPL`) é apontado para o topo da SRAM (`RAMEND`) para suportar chamadas de sub-rotina. Os registradores `RESP`, `A` e `SINAL` são zerados com `CLR`, e `R30` é carregado com `0x0E` (14 em decimal). O laço principal invoca sequencialmente `SHIFT_DOIS` e `CALCULA_SINAL` até que `R30` atinja zero.
2) **Alimentação do Resto Parcial (`SHIFT_DOIS`):** Executa duas vezes a sequência encadeada `LSL INPUT_LOW` → `ROL INPUT_HIGH` → `ROL A_LOW` → `ROL A_HIGH`. Como a instrução `LSL` desloca o bit 7 para a *flag Carry* (C) e preenche o bit 0 com zero, e a instrução `ROL` insere o *Carry* anterior no bit 0 enquanto envia o bit 7 adiante, os 4 registradores comportam-se como um único registrador de deslocamento de 32 bits. Em duas passagens, os 2 bits mais significativos de `INPUT` migram para os 2 bits menos significativos de `A`. Após a 8ª iteração, `INPUT` contém apenas zeros, alimentando automaticamente os pares `00` necessários para a parte fracionária.
3) **Construção do Termo de Teste e Subtração (`CALCULA_SINAL`):** Copia a resposta parcial para `RESP_TEMP` via `MOVW`, desloca o par 2 bits à esquerda e aplica `SBR RESP_TEMP_LOW, 0x01` para fixar o bit 0 em 1, formando $T_k = 4Q_{k-1} + 1$. Em seguida, copia `A` para `SINAL` e efetua a subtração de 16 bits com propagação de empréstimo (`SUB` seguido de `SBC`). A decisão é tomada pela instrução `BRMI NEGATIVO`, que inspeciona a *flag Negative* (N) do registrador de status (`SREG`): se $N = 0$, `A` recebe `SINAL` e `RESP` é

deslocado recebendo bit 1; se $N = 1$, A mantém seu valor anterior e RESP recebe bit 0.

IV. Especificações Técnicas e Custo Computacional

A. Formatos de Dados, Precisão e Limites Representáveis

A Tabela II sintetiza os parâmetros numéricos da entrada e da saída da implementação.

Table II
Especificações Numéricas de Entrada e Saída

Parâmetro Técnico	Entrada (INPUT)	Saída (RESP)
Notação Ponto Fixo	Q16.0 (sem sinal)	Q8.6 (sem sinal)
Largura Registrador	16 bits (R19:R18)	16 bits (R21:R20)
Bits Efetivos	16 bits ($[15:0]$)	14 bits ($[13:0]$)
Bits Int. / Frac.	16 int / 0 frac	8 int / 6 frac
Resolução (1 LSB)	$2^0 = 1$	$2^{-6} = 0{,}015625$
Valor Mínimo	0 (0x0000)	0,0 (0x0000)
Valor Máximo	65535 (0xFFFF)	255,984375 (0x3FFF)

Para converter o valor binário bruto R_{raw} contido em RESP_HIGH:RESP_LOW após a 14ª iteração para sua grandeza real estimada $\hat{y}$, divide-se o inteiro sem sinal por $2^6 = 64$:

$$\hat{y} = \frac{R_{\text{raw}}}{64} = (\texttt{RESP_HIGH} \ll 2) + \frac{\texttt{RESP_LOW}}{64} \tag{6}$$

B. Análise dos Limites Internos e do Desvio Condicional

Uma análise arquitetural detalhada revela duas características importantes decorrentes do uso de registradores internos de 16 bits:

1) **Escolha de 14 Iterações (Q8.6) e Prevenção de Overflow em 16 bits:** A cada iteração k, o registrador RESP acumula k bits significativos. Na subrotina CALCULA_SINAL, o valor de RESP_TEMP é obtido deslocando RESP duas posições à esquerda ($Q_{k-1} \ll 2$). Na 14ª iteração ($k = 14$), Q_{13} possui até 13 bits, de modo que $T_{14} = (Q_{13} \ll 2) \mid 1$ ocupa até **15 bits**, cabendo integralmente no par R27:R26 sem transbordamento. Caso o contador R30 fosse ajustado para 16 iterações (visando saída Q8.8), T_{15} e T_{16} exigiriam 16 e 17 bits respectivamente, causando perda dos bits mais significativos (*overflow*) durante o deslocamento em registradores de 16 bits.
2) **Efeito da Instrução BRMI em Radicandos Elevados:** A instrução BRMI testa a *flag* de sinal N (bit 15 de SINAL_HIGH). Enquanto o bit 15 de A e de SINAL permanece em zero, a subtração em complemento de dois de 16 bits identifica exatamente se $A_k < T_k$. Contudo, para radicandos $X \geq 16384$ (0x4000), nos quais $\sqrt{X} \geq 128$, o bit mais significativo da resposta ($b_1 = 1$) atinge a posição de bit 12 em Q_{13} e, ao ser deslocado 2 vezes na 14ª iteração, ocupa o bit 14 de RESP_TEMP; se simultaneamente o resíduo parcial ou o resultado da subtração alcançar o bit 15 (0x8000), a *flag* N é ativada. Portanto, o uso de BRMI opera sem ambiguidade de sinal em toda a faixa de $0 \leq X < 16384$, podendo ser estendido para toda a faixa de 16 bits sem sinal ($0 \leq X \leq 65535$) com a simples substituição de BRMI por BRCS (*Branch if Carry Set*), que avalia diretamente o empréstimo (*borrow*) sem sinal.

C. Custo Computacional (Ciclos de Clock e Memória)

A avaliação de desempenho no ATmega328 foi conduzida contabilizando os ciclos de máquina de cada instrução AVR:

- **Custo de SHIFT_DOIS:** CALL (4 ciclos) + LDI (1) + 2 iterações de [LSL (1) + 3×ROL (3) + DEC (1) + BRNE (2 na 1ª, 1 na 2ª)] (13 ciclos) + RET (4 ciclos) = **22 ciclos**.
- **Custo de CALCULA_SINAL:** CALL (4) + MOVW (1) + LDI (1) + laço SHIFT_SINAL (9) + SBR (1) + MOVW (1) + SUB (1) + SBC (1) + decisão e retorno (8 ciclos se $S_k \geq 0$; 7 ciclos se $S_k < 0$) = **26 ou 27 ciclos**.
- **Controle do LOOP:** DEC R30 (1) + BRNE (2 ciclos nas 13 primeiras iterações; 1 ciclo na última).

Cada uma das 14 iterações consome entre 51 e 52 ciclos de clock. Somando os 13 ciclos de inicialização da pilha e dos registradores na rotina MAIN, o tempo total de execução situa-se entre **726 e 740 ciclos de clock** (média de 733 ciclos). Operando na frequência padrão de 16 MHz do ATmega328, o cálculo completo leva apenas **45,8 μs**, com variação máxima (*jitter*) de 14 ciclos (0,875 μs), atendendo com folga sistemas de controle em tempo real.

V. Análise do Erro da Estimativa

A. Erro de Truncamento e Viés Sistemático

Como o algoritmo interrompe a extração na 6ª casa fracionária binária sem etapa adicional de arredondamento, a operação realizada é matematicamente equivalente à função piso (*floor*) aplicada sobre a raiz escalonada:

$$\hat{y}(X) = \frac{\lfloor 2^6 \sqrt{X} \rfloor}{2^6} = \frac{\lfloor 64\sqrt{X} \rfloor}{64} \tag{7}$$

Pela desigualdade fundamental da função piso, sabe-se que $\lfloor 64\sqrt{X} \rfloor \leq 64\sqrt{X} < \lfloor 64\sqrt{X} \rfloor + 1$. Dividindo todos os termos por 64, obtém-se o intervalo exato do erro absoluto $e_{\text{abs}}(X) = \hat{y}(X) - \sqrt{X}$:

$$-2^{-6} < \hat{y}(X) - \sqrt{X} \leq 0 \;\Rightarrow\; 0 \leq \sqrt{X} - \hat{y}(X) < 0{,}015625 \tag{8}$$

Conclui-se que:

1) **Ausência de Sobressinal:** A estimativa possui viés exclusivamente não-positivo ($\hat{y}(X) \leq \sqrt{X}$), nunca superestimando a grandeza física medida.
2) **Exatidão em Quadrados Perfeitos:** Para qualquer entrada X cuja raiz quadrada possua representação binária exata com até 6 bits fracionários (incluindo todos os quadrados perfeitos inteiros), o resto parcial zera e o erro é **exatamente nulo** ($e_{\text{abs}} = 0$).
3) **Limitante Superior Estrito:** O erro absoluto máximo é estritamente inferior a 1 LSB da representação Q8.6 ($|e_{\text{abs}}| < 0{,}015625$).

4) **Comportamento do Erro Relativo:** O erro relativo percentual, limitado por $|e_{\text{rel}}| < \frac{0{,}015625}{\sqrt{X}} \times 100\%$ para $X \geq 1$, decai monotonicamente com a raiz do radicando, sendo inferior a $0{,}16\%$ para qualquer $X \geq 100$.

B. Validação Numérica

A Tabela III apresenta a comparação entre o valor real de referência ($\sqrt{X}$) e a saída produzida pelo algoritmo em Q8.6 para diferentes pontos de operação da entrada Q16.0.

Os resultados experimentais confirmam a análise teórica: em todos os casos não exatos, o erro absoluto manteve-se estritamente abaixo do limite de resolução de $0{,}015625$ (atingindo $0{,}015594$ no caso crítico $X = 255$), enquanto entradas quadradas perfeitas (121 e 10000) apresentaram erro zero.

VI. Conclusão

A implementação em Assembly AVR do algoritmo dígito a dígito (*shift-and-subtract*) mostrou-se altamente eficiente para o cálculo da raiz quadrada no ATmega328. Ao substituir divisões e operações em ponto flutuante por deslocamentos lógicos e subtrações de 16 bits, o sistema realiza a conversão de Q16.0 para Q8.6 em cerca de 733 ciclos de clock ($45{,}8\ \mu$s a 16 MHz), utilizando apenas 13 registradores e memória de programa mínima. A análise numérica comprovou que o erro de truncamento é estritamente limitado por $2^{-6} = 0{,}015625$, garantindo previsibilidade temporal e precisão adequada para aplicações de instrumentação embarcada.

References

[1] Atmel Corporation, *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet*, Rev. 7810D-AVR-01/15, 2015.

[2] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. New York: Oxford University Press, 2010.

[3] D. Patterson e J. Hennessy, *Organização e Projeto de Computadores: A Interface Hardware/Software*, 5ª ed. Rio de Janeiro: Elsevier, 2017.

Table III
COMPARAÇÃO ENTRE REFERÊNCIA REAL E ESTIMATIVA DO ALGORITMO (Q8.6)

Entrada X (Q16.0)	**Saída Hex (Q8.6)**	**Estimativa $\hat{y}$ (Decimal)**	**Valor Real $\sqrt{X}$ (Referência)**	**Erro Absoluto $\lvert e_{\text{abs}} \rvert$**	**Erro Relativo $\lvert e_{\text{rel}} \rvert$ (%**
0	0x0000	0,000000	0,000000	0,000000	0,000%
2	0x005A	1,406250	1,414214	0,007964	0,563%
10	0x00CA	3,156250	3,162278	0,006028	0,191%
121	0x02C0	11,000000	11,000000	0,000000	0,000%
255	0x03FD	15,953125	15,968719	0,015594	0,098%
1000	0x07E7	31,609375	31,622777	0,013402	0,042%
10000	0x1900	100,000000	100,000000	0,000000	0,000%
16383	0x1FFF	127,984375	127,996094	0,011719	0,009%